

Top Twenty Tips for Using Generative AI in Systems Engineering

Ray Madachy
Naval Postgraduate School
Systems Engineering Dept.
rjmadach@nps.edu

Background

This presentation is based on the following INCOSE INSIGHT special issue article with additions:

Ray Madachy, Dan O'Leary, Tom McDermott, and Sinan Bank. *Top Twenty Tips for Using Generative AI in Systems Engineering. INCOSE INSIGHT, June 2026.*
https://incose.onlinelibrary.wiley.com/doi/10.1002/inst.70053



1. Start with a clear purpose and a way to measure success.

- Specify the decision, deliverable, and intended reviewer.
- Define success criteria before generating a response.
- Use small experiments to identify worthwhile applications.

Example
Suppose a team wants to use AI to draft subsystem requirements. Start with ten representative source statements and a human-reviewed reference set. Define measures such as valid requirements produced, defects found during review, review time, and rework. Compare the assisted workflow against the current approach before extending it.

2. Use AI as part of a larger engineering process.

- Use AI for interpretation, drafting, comparison, and synthesis.
- Keep models, databases, requirements tools, and human reviews.
- Define test inputs and expected outputs for each AI-assisted step.

Example
In a verification planning process, AI could read a set of requirements and suggest candidate test cases. Those cases are then examined by the verification lead, linked to requirement identifiers in the authoritative management system, and checked for feasibility. The official traceability matrix stays in the controlled tool rather than living only in a chat.

3. Treat prompting like engineering.

- State the objective, assumptions, and operating constraints.
- Specify output structure and acceptance criteria.
- Save and improve prompts that work reproducibly.

Example
Instead of asking for a trade study, request a comparison of three power-system architectures against named criteria. Supply the mission duration, power demand, cost boundaries, units, and a required decision matrix. Direct the AI to label missing data, avoid invented numerical estimates, and state which criteria require external analysis. Test the prompt on a known case before reuse.

4. Manage context deliberately across sessions and keep a record of AI-assisted work.

- Record inputs, sources, assumptions, outputs, and decisions.
- Preserve accepted artifacts and rejected alternatives.
- Reestablish the approved baseline when work resumes.

Example
A team drafting an interface control document maintains a repository entry with the approved interface definition, the source documents, the prompt template version, the generated proposal, and reviewer comments. On the next session, the engineer attaches the current baseline and asks the model to identify only the differences. The process avoids rebuilding decisions from memory.

5. Let AI ask questions before it answers.

- Ask AI to surface gaps, ambiguity, and conflicts.
- Require questions when constraints are insufficient.
- Answer critical questions before requesting the final artifact.

Example
A customer asks for an autonomous vehicle that is easy to maintain and deploys rapidly. Before drafting requirements, the AI asks about turnaround time, maintenance environment, operator skill, mission duration, and the operational definition of deployment. The engineer can answer or mark unknowns for resolution. Only then does the model draft measurable requirements.

6. Ask for structured outputs.

- Request tables, schemas, matrices, code, or checklists.
- Retain identifiers and links to authoritative artifacts.
- Request rationale separately from the structured result.

Example
Ask the AI to generate a verification matrix with columns for requirement identifier, proposed method, test resource, pass criterion, source reference, and unresolved issue. Validate that every expected requirement ID appears, that fields use controlled vocabulary, and that source references exist. Import only reviewed entries into the project database.

7. Use clear rubrics and evaluation criteria.

- Define completeness, consistency, compliance, and coverage.
- Use pass/fail tests where objective checks are possible.
- Separate AI recommendations from engineer acceptance.

Example
For each drafted requirement, ask the AI to return a pass/fail value for each defined writing criterion, an explanation, and the text span that triggered a failure. Reviewers can then sample both passes and failures and compare their judgments with the model. The result is a repeatable quality-assurance procedure rather than an unsupported overall score.

8. Be transparent about sources and uncertainty.

- Distinguish supplied documents, retrieval, and general knowledge.
- Request source references and explicit assumptions.
- Flag unsupported claims and what would resolve them.

Example
An AI assistant recommends a thermal-management design. Ask it to tabulate each claim, the source document and section, the assumption used, and the remaining uncertainty. If the heat-load data are not in the baseline, the table should say that the design recommendation is provisional. The engineer can then seek the missing calculation or measurement.

9. Learn how retrieval-augmented generation (RAG) works.

- Understand chunking, embeddings, and retrieval ranking.
- Check whether the right source passages are actually retrieved.
- Maintain retrieval collections as governed engineering assets.

Example

Build a small RAG collection from approved interface control documents. Ask a set of known interface questions, inspect the passages returned before generation, and record when the correct section was missed. Try alternative chunk sizes or metadata filters, then compare retrieval success before evaluating the generated answer. Keep baseline revisions identifiable.

10. Build and govern your source data carefully.

- Curate current, complete, authoritative documents.
- Rebuild retrieval indexes after relevant source changes.
- Protect sensitive and export-controlled information.

Example
A systems team revises a subsystem specification. The document owner archives the prior revision, updates the approved corpus, rebuilds the retrieval index, and runs regression questions to confirm that the assistant now cites the new values. A separate access-control check determines whether a controlled document is allowed in the chosen environment.

11. Stay current.

- Periodically compare new models, tools, and methods.
- Retest previously difficult engineering use cases.
- Avoid perpetuating obsolete technical workarounds.

Example
Six months ago, a requirements comparison task may have failed because long documents were truncated or key references were missed. When a new model or retrieval pipeline becomes available, rerun the same controlled evaluation set. Compare accuracy, omissions, review effort, execution time, and cost before changing the standard workflow.

12. Choose models based on evidence, not hype.

- Benchmark on representative tasks and failure modes.
- Compare quality, latency, cost, context, and privacy.
- Consider small local models where they meet the need.

Example
Prepare twenty requirements containing known ambiguities and twenty acceptable requirements. Run several candidate models under controlled prompting and compare detection rates, false alarms, rationale quality, execution time, and data-handling suitability. Repeat using local execution if organizational policy makes cloud use difficult. Select a model by the evidence and constraints.

13. Treat deployment and data constraints as part of the task.

- Identify classification, privacy, IP, and export restrictions.
- Choose public cloud, secure cloud, local, or isolated execution.
- Verify data handling and organizational policy before use.

Example
For a design review involving proprietary supplier details, the team first identifies approved data locations and allowed AI services. It may use a local model for the sensitive document and a cloud model for non-sensitive generic drafting. Both workflows document the boundary and prevent material from crossing it in prompts, retrieval stores, or logs.

14. Build for change.

- Separate prompts, configuration, retrieval, and tool interfaces.
- Maintain regression cases and known-good baselines.
- Swap models or environments without rewriting the process.

Example
A cost-estimation assistant accepts a defined JSON input and returns a fixed result schema. The language model interprets the user request, but the deterministic estimator is invoked through a stable tool interface. When the model is upgraded or moved from cloud to local execution, the team reruns the same tests without redesigning the cost model.

15. Understand where AI works and where it doesn't.

- Test narrow use cases before generalizing results.
- Characterize omissions, unsupported claims, and variance.
- Set boundaries for tasks requiring formal methods or tools.

Example
A model extracts failure modes correctly from one FMEA worksheet. Before relying on it for another worksheet, test difficult formats, missing fields, conflicting descriptions, and rare failure categories. Check whether errors are consistent and whether the system signals uncertainty. Do not infer that success at extraction proves it can compute system reliability.

16. Spend more time verifying than generating.

- Pilot on small cases with a known reference answer.
- Measure false positives, false negatives, and rework.
- Allocate human effort to verification, validation, and curation.

Example
An assistant creates one hundred draft test cases. Instead of accepting all one hundred, the verification team evaluates a representative batch against the requirements, identifies missing scenarios and invalid acceptance limits, and measures review time. Only when the observed defect rates are acceptable does the team expand the workflow, with continued sampling and human approval.

17. Don't trust confidence estimates until you test them.

- Treat self-reported confidence as uncalibrated.
- Compare predictions to ground truth across many cases.
- Use calibrated scores only after empirical validation.

Example
Collect AI judgments on fifty requirements with known acceptance outcomes. Group them into stated-confidence bands and calculate the observed correctness in each band. If the model says 90 percent for many erroneous cases, the score must not be used as an automatic release criterion. Prefer externally measured, calibrated performance and continued monitoring.

18. Pair agent reasoning with deterministic tools.

- Use AI to frame questions, select tools, and explain results.
- Delegate simulation, calculation, and validation to governed tools.
- Check tool inputs, outputs, and execution records.

Example

An engineer asks an assistant to estimate effort for a system concept. The assistant clarifies the drivers, formats the inputs for an approved systems engineering cost model, invokes the model, and reports the returned estimate with inputs and assumptions. It may explain sensitivity, but it must not invent numerical results or claim to have run a tool that was not called.

19. Design workflows that allow the AI to abstain.

- Define when the AI must defer instead of guessing.
- Route ambiguous or high-risk results to human review.
- Make uncertainty and escalation part of the output schema.

Example
A compliance-review agent cannot locate the governing paragraph for a safety-related requirement. Instead of labeling it compliant based on general patterns, it returns status NEEDS REVIEW, lists the missing source, and assigns a human reviewer. The workflow records the unresolved item and prevents it from entering the approved set.

20. Focus on human-AI teaming and continuous improvement.

- Keep accountable engineers in the decision loop.
- Measure prompt effort, review effort, errors, and rework.
- Update prompts, tools, data, and procedures from evidence.

Example
At the end of each design iteration, a team reviews metrics for AI-generated requirements: acceptance rate, false omissions, average review time, defect categories, and rework hours. They revise the prompt template and source corpus, then rerun the validation set. The next iteration tests whether the process actually improved rather than assuming that it did.